

Customer Churn Analytics - MCP Server

AI Engineer Home Assignment | Dataset: aai510-group1/telco-customer-churn (Hugging Face) | Stack: Python + DuckDB + MCP

DESIGN - WHAT IT IS AND HOW IT WORKS

The server exposes the churn dataset as six MCP tools backed by an in-memory **DuckDB** database. `ask_question()` is the primary tool: it runs a dedicated local LLM (Ollama) that converts a natural-language question straight into SQL and returns the answer - demonstrating the server's own model integration rather than relying solely on the host LLM. The other five are power-user / fallback tools for direct SQL access.

Tool	Purpose
ask_question(q)	PRIMARY - NL to SQL via local Ollama, free
get_schema()	Power-user: column names + types
get_sample_rows(n)	Power-user: inspect values and encoding
query_sql(sql)	Power-user: run any SELECT / WITH directly
get_churn_summary()	Pre-computed KPIs in one shot
get_column_distribution(col)	Value counts or numeric stats

WHY SQL + DUCKDB (NOT RAG)

Churn data is **structured and tabular**. Questions like "What percent of fiber-optic customers on month-to-month contracts churn?" need exact arithmetic - not semantic search. SQL gives **deterministic, auditable answers**; RAG would hallucinate aggregates and adds chunking complexity for zero benefit on a flat CSV.

DuckDB needs no server, reads CSVs natively with `read_csv_auto`, supports advanced SQL (MEDIAN, PERCENTILE_CONT, PIVOT), and runs in-memory - zero infrastructure for a local MCP server.

Deliberately avoided: vector embeddings, LangChain, a running DB server. Each adds complexity without improving accuracy on structured data.

PRODUCTION CHANGES

- Swap DuckDB for **BigQuery / Snowflake** - TB-scale, multi-user concurrency
- Add **OAuth / API-key auth** so only authorized users can query
- **Cache** common queries (Redis) - churn summaries don't change by the minute
- **Scheduled ingestion** from CRM/warehouse; version datasets with timestamps
- **Observability:** log every SQL query, latency, and user ID for audit and cost
- Rate-limit `ask_question`; add **prompt-injection guardrails** on NL input

RISKS AND MITIGATIONS

Risk	Mitigation
SQL injection via <code>query_sql</code>	Allowlist SELECT/WITH; read-only DuckDB
NL-to-SQL mistranslation	<code>ask_question</code> returns SQL for user review
Stale / drifted data	Versioned ingestion; freshness metadata tool
Column name changes	Schema fetched first; case-insensitive match
LLM hallucination on stats	Tools return raw numbers - no guessing
Local model unavailable	<code>ask_question</code> fails gracefully w/ setup steps

BUSINESS IMPACT AT CHEQ

CHEQ's mission is ensuring marketing budgets reach **real, high-intent humans**. Invalid traffic (bots, click-fraud) corrupts the customer data that churn models depend on - a bot that clicks an ad but never converts looks exactly like a churned real customer. This MCP server creates value at three levels:

- **Internal retention:** CHEQ's CS team asks which customers are at-risk and gets SQL-backed answers instantly - no analyst needed, decisions in minutes.
- **Product intelligence:** Correlate churn with usage signals to identify *feature gaps* driving cancellations and inform the product roadmap.
- **Differentiated offering:** Package this MCP pattern for CHEQ customers - let them query their own fraud-cleaned audience data in natural language. CHEQ's invalid-traffic filtering makes churn signals *trustworthy*; the MCP layer makes them *accessible* to non-technical stakeholders.
- **Faster experimentation:** Marketing and CX teams test retention hypotheses on payment method, contract type, or fiber optic without waiting for BI sprints.

EXAMPLE QUESTIONS ANSWERED

Q: "What is the overall churn rate?"
-> `get_churn_summary()` returns exact rate instantly
Q: "Which contract type retains best?"
-> Two-Year contracts churn far less than Month-to-Month
Q: "How much revenue is at risk from churn?"
-> `query_sql()` sums Monthly Charge for churned segment
Q: "What are top reasons customers cite for churning?"
-> `query_sql()` groups by Churn Category / Churn Reason

Repo: github.com/ShaiNa111/CHEQ-Home-Assignment | **Model:** qwen2.5-coder:14b via Ollama, local & free (primary, `ask_question`) | **Stack:** Python 3.11 - mcp 1.x - DuckDB - pandas